GitHub Copilot
How It Works, Context, and Prompt Crafting

Learning Objectives

By the end of this lesson, you should be able to:

- Explain the basic workflow behind GitHub Copilot code suggestions.
- Describe the role of **context** in AI-assisted programming.
- Explain how information from your development environment can influence Copilot suggestions.
- Distinguish between project data and the context selected for a particular Copilot interaction.
- Explain the purpose of **prompt crafting**.
- Apply four important prompt-crafting principles:
    1. Context
    2. Intent
    3. Clarity
    4. Specificity
- Improve vague programming prompts so that they produce more relevant code suggestions.
- Evaluate Copilot suggestions rather than treating generated code as automatically correct.

Section 1 — How GitHub Copilot Works

1.1 Why Understand the Process?

Before concentrating on individual GitHub Copilot features, it is useful to understand what happens when Copilot assists you with code.

Knowing how to operate the tool is only one part of using it effectively.

A developer should also understand:

- what information Copilot uses;
- how that information affects its suggestions;
- why some suggestions are more relevant than others;
- why Copilot may misunderstand a task;

- how the developer can improve the information provided to Copilot.

This leads to an important principle:

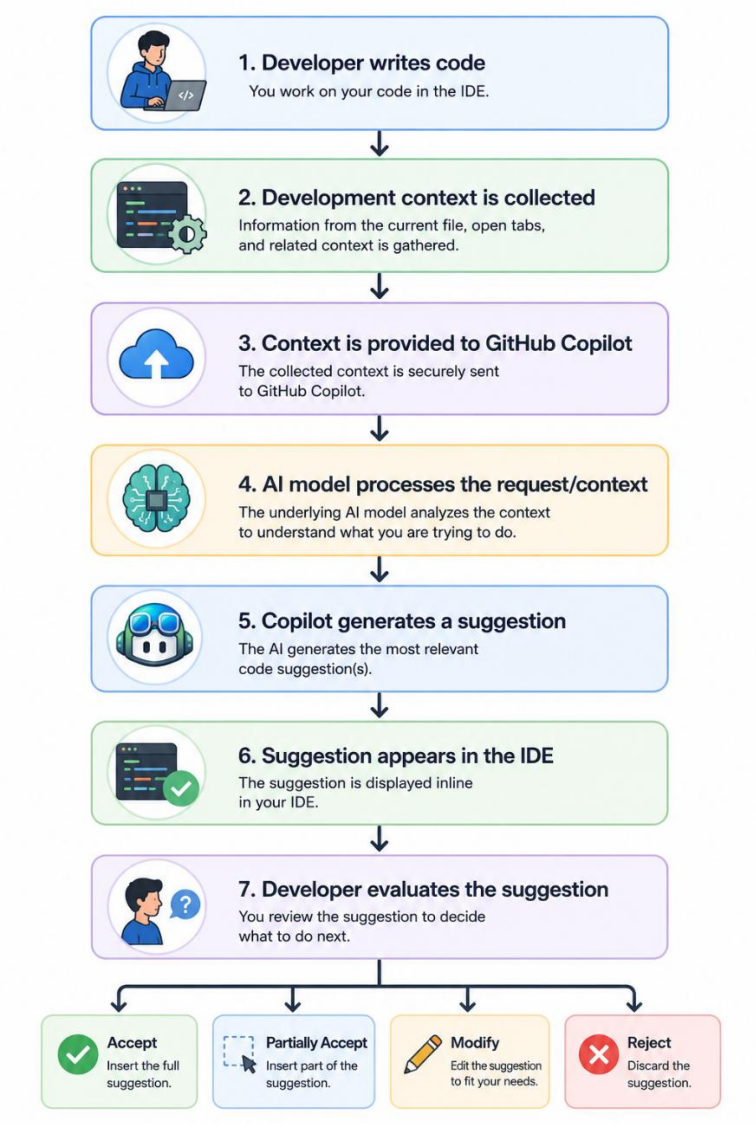
Better input and better context generally make useful output more likely.

Understanding this process allows us to use Copilot deliberately rather than simply waiting for it to generate code.

1.2 The Basic Copilot Workflow

Consider a developer working on a piece of code inside an IDE such as Visual Studio Code.

At a high level, the interaction can be represented as:



The key point is that this is an **interactive development cycle**.

Copilot does not independently develop the application. It participates in the developer's existing workflow.

1.3 Step-by-Step: What Happens While You Code?

Let's examine the process more closely.

Step 1 — You Work on Your Code



Suppose you are writing a Java method:

```
public double calculateAverage(List<Integer> values) {
```

You have not explicitly told Copilot every implementation detail.

However, the code already communicates useful information.

For example:

- the language is Java;
- the method returns a **`double`**;
- the method name is **`calculateAverage`**;
- the input is a **`List<Integer>;`**
- the likely task is to calculate an average.

This information forms part of the context that can help Copilot determine what you are trying to accomplish.

Step 2 — Copilot Uses Context



Copilot uses relevant context from your development environment when producing suggestions.

Conceptually:

```
Your Code
   +
Comments
   +
Relevant Development Context
   ↓
GitHub Copilot
```

For example, adding a useful comment may provide additional information:

```
// Calculate the arithmetic mean.
// Return 0 if the list is empty.
public double calculateAverage(List<Integer> values) {
```

The comment introduces an important requirement:

```
Empty list → return 0
```

Without that information, Copilot would have to infer what should happen when the list is empty.

TIP — Make Intent Visible

Good names, useful comments, clear method signatures, and well-structured code are useful not only for human developers. They also provide better signals to AI coding assistants.

Step 3 — The Model Processes the Context



Copilot uses AI models to process the available context and generate a likely continuation or response.

Conceptually:

```
Context
    ↓
AI model
    ↓
Predicted useful code
```

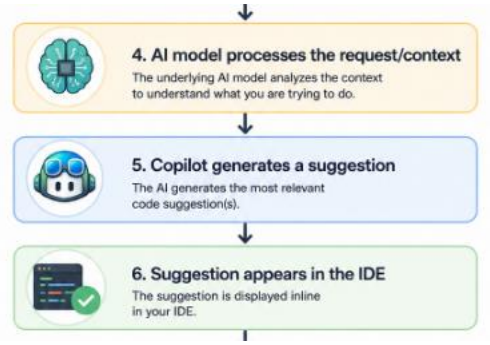
The important word here is **predicted**.

Copilot is not executing your requirements and mathematically proving that the generated implementation is correct. It is generating an output based on the information available to it.

Therefore:

AI-generated code ≠ guaranteed correct code

Step 4 — Copilot Returns a Suggestion to the IDE



The generated suggestion is presented directly in your development environment.

This integration is one of Copilot's main productivity advantages.

Instead of:

```
Stop coding
    ↓
Open browser
    ↓
Search for syntax/example
    ↓
Read several results
    ↓
Find relevant code
    ↓
Return to IDE
    ↓
Continue coding
```

the workflow can become:

```
Continue coding
    ↓
Receive Copilot suggestion
    ↓
Evaluate suggestion
    ↓
Continue development
```

1.4 Maintaining Development Flow

Developers frequently perform small searches while programming.

For example:

- How do I sort this collection?
- What is the syntax for this API?
- How do I iterate through this object?
- How do I convert this value?
- What method does this library provide?
- What exception should I handle?

Even a two-minute search can interrupt concentration.

Copilot attempts to reduce this **context switching** by bringing assistance directly into the IDE.

NOTE — Developer Flow

One practical advantage of an IDE-integrated AI assistant is not simply that it can generate code. It can reduce the number of times developers must leave their current development environment for routine questions.

1.5 The Developer Remains in Control

When Copilot generates a suggestion, you are not required to accept it.

Depending on the interface and suggestion, you may be able to:

- accept the suggestion;
- reject it;
- accept only part of it;
- modify the generated code;
- request another solution;
- use the suggestion as the starting point for your own implementation.

The appropriate workflow is:

```
Copilot suggests
       ↓
Developer evaluates
       ↓
Developer decides
```

not:

```
Copilot suggests
       ↓
Automatically assume correctness
```

WARNING — Suggestion Is Not Validation

A Copilot suggestion may look professional and syntactically correct while still containing logical, architectural, performance, or security problems.

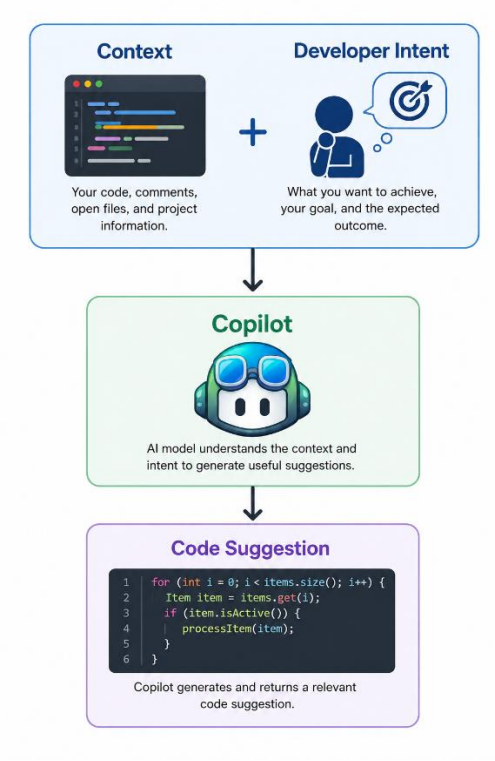
The developer remains responsible for reviewing and testing generated code.

Section 2 — How GitHub Copilot Uses Context

2.1 What Is Context?

In AI-assisted programming, **context** refers to information made available to the AI so that it can understand the current task and produce an appropriate response.

A simplified model is:



Consider this incomplete statement:

```
result =
```

There is very little information available.

Compare it with:

```
// Calculate the total price including 15% tax
double result =
```

The second example communicates significantly more information about the intended operation.

2.2 The Current File as Context

The file you are actively editing is an important source of context.

Suppose the current file contains:

```
public class EmployeeService {

    private EmployeeRepository repository;

    public Employee findEmployeeById(long id) {
        // implementation
    }
}
```

Copilot can infer useful information from elements such as:

- class name: **`EmployeeService;`**
- dependency: **`EmployeeRepository;`**
- method name: **`findEmployeeById;`**
- parameter: **`long id;`**
- return type: **`Employee`**.

The surrounding code helps establish what kind of implementation is appropriate.

This is why code quality can affect AI assistance.

Compare:

```
public Object doIt(Object x) {
```

with:

```
public Employee findEmployeeById(long employeeId) {
```

The second version communicates much more semantic information.

TIP — Descriptive Code Creates Better Context

Use meaningful class, method, parameter, and variable names. This improves maintainability for humans and also gives Copilot stronger information about the intended behavior.

2.3 Open Files and Related Context

Depending on the Copilot feature, IDE integration, model, and current product behavior, Copilot can use additional relevant workspace information beyond the exact line currently being edited.

Open and related files can therefore provide useful contextual signals.

For example, imagine working on:

`EmployeeService.java`

while related files include:

`Employee.java`

`EmployeeRepository.java`

`EmployeeController.java`

Information from these related artifacts may help Copilot understand:

- the domain model;
- available methods;
- naming conventions;
- relationships between components;
- expected data types;
- project structure.

2.4 Think in Terms of Relevant Context, Not the Entire Project

A useful conceptual model is that Copilot tries to work with **relevant context** rather than treating every byte in a large project as equally important.

Consider a project containing:

5,000 files

but your current task is:

Implement **`EmployeeService.findEmployeeById()`**

Sending or treating every file as equally relevant would be inefficient.

An analogy is asking someone a question about a few pages of a very large book.

Inefficient approach

"Here is a 5,000-page book.

Read everything and answer my question about page 217."

More focused approach

"Here are the pages relevant to the question.

Use them to answer it."

The second approach reduces irrelevant information.

NOTE — Context Selection Is Feature-Dependent

Copilot's exact context-selection behavior can vary by feature, IDE integration, model, and product version. Avoid assuming a fixed rule such as “Copilot always sends exactly the current file plus every open tab.”

For practical use, the important concept is that **relevant accessible context influences the generated result**.

2.5 Context Proximity

Information closely related to the code you are currently working on is generally more useful than unrelated information.

Conceptually:

```
Current task
     ↓
Current code
     ↓
Closely related code
     ↓
Other relevant project information
```

Suppose you are implementing:

```
public EmployeeDTO getEmployee(long id)
```

The following information may be highly relevant:

```
EmployeeDTO
Employee
EmployeeRepository
```

```
Existing mapping methods
Related service methods
```

An unrelated file such as:

```
InvoicePdfGenerator.java
```

is unlikely to help unless the current task actually depends on it.

2.6 Practical Exercise — Improve Context

Consider this code:

```
public List getData() {
```

From Copilot's perspective, many questions remain:

- What kind of data?
- From where?
- What should the list contain?
- Should errors be handled?
- Should results be filtered?
- Should they be sorted?

Now improve the code:

```
// Retrieve all active employees from the repository,
// sort them alphabetically by last name,
// and return them as EmployeeDTO objects.
public List<EmployeeDTO> getActiveEmployees() {
```

The second version provides considerably stronger context.

Step 1

Identify the operation:

```
Retrieve employees
```

Step 2

Specify the condition:

Active employees only

Step 3

Specify ordering:

```
Sort by last name
```

Step 4

Specify output:

```
List<EmployeeDTO>
```

This gives Copilot a clearer description of the intended behavior.

PITFALL — Expecting Copilot to Read Your Mind

Developers sometimes provide minimal information and expect a highly specific implementation.

If an important requirement exists only in your head, Copilot may not know about it. Make important constraints visible through the prompt, code, comments, types, or other relevant context.

Section 3 — Prompt Crafting

3.1 What Is Prompt Crafting?

Prompt crafting is the process of designing and refining instructions so that an AI system has sufficient information to generate useful output.

For GitHub Copilot, prompts may appear in several forms, including:

- natural-language questions;
- Copilot Chat requests;
- comments;
- method descriptions;
- existing code;
- instructions describing a desired implementation.

A useful prompt communicates both:

What are we doing?

and:

What result do we expect?

3.2 Why Prompt Quality Matters

Compare these prompts.

Prompt A

Write code.

This provides almost no useful information.

Prompt B

Write a Java method that calculates the average of a list of integers.

Return 0 if the list is empty.

Prompt B specifies:

- language;
- artifact type;
- operation;
- input concept;
- special-case behavior.

Therefore, Copilot has fewer important details to infer.

A useful relationship is:

```
Vague Prompt
      ↓
Many Possible Interpretations
      ↓
Potentially Generic Output
```

versus:

```
Well-Crafted Prompt
         ↓
Fewer Ambiguities
         ↓
More Focused Output
```

3.3 Four Elements of Effective Prompts

For this lesson, we will organize prompt crafting around four elements:

Element	Main Question
Context	What are we building or working with?
Intent	What are we trying to accomplish?
Clarity	Can the instruction be understood unambiguously?
Specificity	What exact requirements and constraints must be satisfied?

A useful mental model is:


A useful mental model
Context
The background and environment your request is based on.
Example: language, framework, project, existing code, constraints.
+
Intent
The goal you want to achieve.
Example: what you want Copilot to help you do.
+
Clarity
Clear and unambiguous instructions that Copilot can understand.
Example: avoid abbreviations, state things plainly.
+
Specificity
The exact details and requirements of what you want.
Example: inputs, outputs, constraints, edge cases, formats, libraries.
Better-defined request
Combining context, intent, clarity, and specificity produces a well-formed, easy-to-understand request.
More relevant Copilot suggestion
Copilot has what it needs to generate code that is more accurate and useful.

3.4 Element 1 — Context

Context tells Copilot about the environment surrounding the task.

It answers questions such as:

- What language are we using?
- What framework are we using?
- What component are we implementing?
- What data structures already exist?
- What libraries are available?
- What conventions does the project follow?

Consider:

Create an endpoint.

This lacks context.

Compare:

In a Spring Boot REST application, create an endpoint

for retrieving an employee by ID.

Now Copilot knows:

Language ecosystem → Java

Framework → Spring Boot

Architecture → REST

Resource → Employee

Operation → Retrieve by ID

TIP — Give Copilot the Information You Would Give Another Developer

Imagine assigning the task to a teammate who understands programming but has not been following your thoughts. What information would that developer need? Much of the same information is useful when prompting Copilot.

3.5 Element 2 — Intent

Intent communicates the goal you are trying to achieve.

Consider:

Use EmployeeRepository.

It identifies a component but does not clearly communicate the goal.

Compare:

Use EmployeeRepository to retrieve an employee by ID

and return the employee to the service layer.

The intent is now much clearer.

Intent may also communicate architectural expectations.

For example:

Implement this using the service layer rather than accessing

the repository directly from the controller.

This tells Copilot something about the application's design.

Intent can help generated code align with:

- architecture;
- coding conventions;
- design patterns;
- application behavior;
- maintainability requirements.

3.6 Element 3 — Clarity

A prompt should be easy to interpret.

Avoid unnecessary ambiguity, unexplained abbreviations, or vague statements.

Unclear

Do the emp thing and return it.

A human teammate may not know what this means. Neither should we expect an AI assistant to infer the intended operation reliably.

Clear

Retrieve the employee with the specified ID and return

an EmployeeDTO.

Another example:

Unclear Prompt

> *Write a loop.*

- What kind of loop?
- For what collection?
- What should happen during each iteration?
- Which programming language?

Clear Prompt

> *Write a JavaScript for loop that iterates over an array*
>
> *of objects and prints each object's name property*
>
> *to the console.*

The second prompt answers the important questions.

PITFALL — Personal Abbreviations

Avoid project shorthand or abbreviations that have not been established in the available context.

Something obvious to you may be ambiguous to Copilot.

3.7 Element 4 — Specificity

Specificity identifies the exact details of the desired solution.

Consider the following prompt:

Retrieve data from a database.

This leaves many unanswered questions:

- Which language?
- Which database?
- Which library?

- Which table?
- Which rows?
- Which columns?
- What return format?
- How should errors be handled?

A more specific prompt would be:

> *Write a Python function that connects to a PostgreSQL database*
>
> *using the specified database library.*
>
> *Query all rows from the employees table.*
>
> *Return the result as a list of dictionaries, where each dictionary*
>
> *represents one employee and uses the column names as keys.*

Now the requirements are much more constrained.

3.8 Building a Prompt Step by Step

Instead of trying to create a perfect prompt immediately, construct it incrementally.

Suppose we begin with:

Get employees.

Step 1 — Add Context

In a Python application using PostgreSQL, get employees.

Step 2 — Add Intent

In a Python application using PostgreSQL,

retrieve all employees from the employees table.

Step 3 — Improve Clarity

Write a Python function that connects to PostgreSQL

and retrieves all rows from the employees table.

Step 4 — Add Specificity

Write a Python function that connects to a PostgreSQL database

using the specified database library.

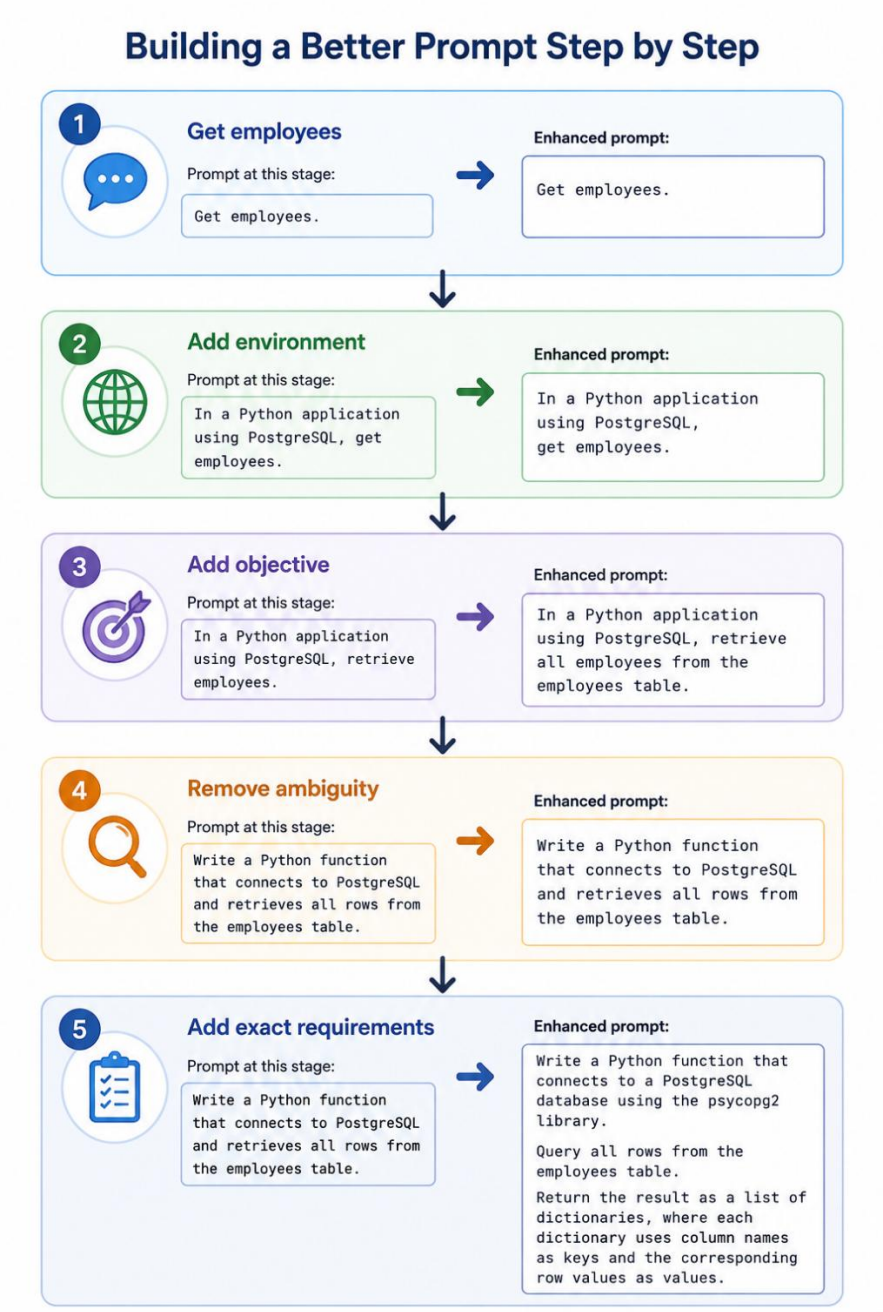
Query all rows from the employees table.

Return the rows as a list of dictionaries.

Each dictionary should use database column names as keys

and the corresponding row values as values.

The progression is:



3.9 Prompt-Crafting Template

For programming tasks, the following structure can be useful:

For example:

I am working with Java and Spring Boot.

I need a service method that retrieves an employee by ID.

The method should:

- accept a long employee ID;

- use EmployeeRepository;

- return an EmployeeDTO;

- throw EmployeeNotFoundException if the employee does not exist.

Use the existing EmployeeMapper to convert Employee to EmployeeDTO.

Notice that this prompt provides:

Context → Java + Spring Boot

Intent → Retrieve employee

Clarity → Explicit service operation

Specificity → ID, repository, DTO, exception, mapper

Prompt Template for GitHub Copilot

Use this structure to write clear, complete, and effective prompts.



[Context]

Provide the background and environment relevant to your request.

I am working with _____.



[Intent]

Clearly state the goal or what you want to achieve.

I need to _____.



[Requirements]

List the key requirements and what the solution should accomplish.

The solution should:

- _____
- _____
- _____



[Constraints]

Specify technologies, libraries, rules, limitations, or things to avoid.

Use _____.

Do not _____.



[Expected Output]

Define the format or type of output you expect.

Return / generate _____.



Why this structure works

It ensures your prompt gives Copilot enough information to understand your task and provide accurate, relevant, and complete code suggestions.

- Clear and organized
- Covers all important aspects
- Improves the quality of suggestions
- Saves time and reduces revisions

3.10 Practical Lab — Improve Weak Prompts

For each prompt below, identify what is missing and rewrite it using the four prompt-crafting principles.

Exercise 1

Weak prompt:

Write a loop.

Ask yourself:

1. Which programming language?
2. What collection?
3. What should happen during iteration?
4. What output is expected?

Improved example:

Write a JavaScript for loop that iterates over an array

of employee objects and prints each employee's name property

to the console.

Exercise 2

Weak prompt:

> Get database data.

Identify the missing information:

> Database?
> Programming language?
> Library?
> Table?
> Query?
> Return type?

Then create a more complete prompt.

Exercise 3

Weak prompt:

> Create employee API.

Improve it by specifying:

- framework;
- HTTP method;
- URL;
- input;
- output;
- service dependency;
- behavior when the employee does not exist.

For example:

> In a Spring Boot REST application, create a GET endpoint
>
> at /api/employees/{id}.
>
> Use EmployeeService to retrieve the employee.
>
> Return EmployeeDTO with HTTP 200 when the employee exists.
>
> Return HTTP 404 when no employee exists with the specified ID.

3.11 A Practical Prompt Review Checklist

Before submitting an important prompt, ask:

Context

- Does Copilot know the language?
- Does it know the framework or library when relevant?
- Does it understand the surrounding application?

Intent

- Have I clearly stated what I want to accomplish?
- Is the purpose of the generated code obvious?

Clarity

- Are my instructions understandable?
- Have I removed ambiguous terminology?
- Have I avoided unexplained abbreviations?

Specificity

- Have I specified important inputs?
- Have I specified expected outputs?
- Have I included constraints?
- Have I described important edge cases?
- Have I identified required technologies or APIs?

TIP — Don't Add Detail Just for the Sake of Detail

A good prompt is not necessarily the longest prompt. Include information that affects the solution. Irrelevant detail can make the task harder to interpret rather than easier.

3.12 Prompt Crafting Is Iterative

You do not always need to provide every requirement in the first prompt.

AI-assisted development can be iterative.

For example:

Prompt 1

> *Write a Java method that searches a list of employees by department.*

Review the result.

Then refine it:

Prompt 2

> *Modify the method so that the department comparison is case-insensitive.*

Then:

Prompt 3

> *Return the employees sorted alphabetically by last name.*

Then:

Prompt 4

Rewrite the implementation using the Java Stream API.

This produces an interaction such as:

```
Prompt
   ↓
Suggestion
   ↓
Review
   ↓
Refine requirements
   ↓
New prompt
   ↓
Improved suggestion
```

NOTE — Prompting Is Part of the Development Cycle

Do not think of prompting as a one-time request for a finished solution. For complex tasks, it is often more effective to generate, inspect, refine, and iterate.

3.13 What a Good Prompt Cannot Guarantee

Even an excellent prompt does not guarantee correct code.

A prompt may contain:

✓ Context

✓ Intent

✓ Clarity

✓ Specificity

and Copilot can still generate:

✗ Incorrect logic

✗ Invalid assumptions

✗ Inappropriate API usage

✗ Missing edge cases

✗ Inefficient code

✗ Security problems

Therefore, prompt quality improves the **quality of the request**, but does not eliminate the need for software engineering practices.

The complete workflow remains:

```
Craft Prompt
      ↓
Generate Suggestion
      ↓
Understand Code
      ↓
Review Code
      ↓
Modify if Necessary
      ↓
Compile / Run
      ↓
Test
      ↓
Debug and Refine
```

WARNING — Better Prompt ≠ Guaranteed Correct Code

Prompt engineering improves the probability of obtaining relevant output. It does not convert AI-generated code into verified software.

Verification remains the developer's responsibility.

---

4. Putting Everything Together

The three concepts covered in this lesson are closely connected.

4.1 Copilot Workflow

```
Developer activity
        ↓
Context
        ↓
AI processing
        ↓
Suggestion
```

```
        ↓
Developer evaluation
```

4.2 Context

Context helps Copilot understand:

- What am I working on?
- What code surrounds this task?
- What technologies are involved?
- What information is relevant?

4.3 Prompt Crafting

Prompt crafting helps communicate:

- What do I want?
- Why do I want it?
- What exactly should it do?
- What constraints must it follow?

Together:

```
Relevant Context
       +
Clear Prompt
       ↓
Better-defined task
       ↓
Potentially more relevant suggestion
       ↓
Developer review and testing
```

---

5. Key Takeaways

1. **GitHub Copilot uses development context to generate code suggestions.**
2. **Context is fundamental to the quality and relevance of AI-assisted programming suggestions.**
3. **Meaningful code structure and naming help communicate developer intent.**
4. **Copilot can reduce context switching by providing assistance directly within the IDE.**

5. **Prompt crafting is the process of deliberately constructing instructions for an AI assistant.**
6. Four useful prompt-crafting principles are:

   Context + Intent + Clarity + Specificity
7. **Vague prompts create more room for interpretation.**
8. **Specific prompts communicate requirements and constraints more effectively.**
9. **Prompt crafting can be iterative.** Generate, review, refine, and ask again when necessary.
10. **The developer remains responsible for generated code.** Copilot output should be understood, reviewed, compiled, tested, and refined before being relied upon.

---

6. Quick Reference

Principle	Question to Ask	Example
Context	What am I working with?	"In a Spring Boot application..."
Intent	What am I trying to achieve?	"Retrieve an employee by ID..."
Clarity	Is the request unambiguous?	"Return an EmployeeDTO..."
Specificity	What exact requirements apply?	"Return 404 if the employee does not exist..."

A useful formula to remember is:

```
Context
   +
Intent
   +
Clarity
   +
Specificity
   ↓
Strong Prompt
   ↓
Relevant Copilot Suggestion
   ↓
Human Review
   ↓
Testing and Validation
```